AI Production Asset Workflow

YouTube Channel — Animated/Illustrated Video Production Pipeline

Purpose: This document defines the end-to-end AI-assisted pipeline used to generate all visual production assets (characters, environments, sprite sheets, and per-beat image prompts) from a script, for handoff to the video editor. The goal of this system is to produce output indistinguishable from a professional production studio — not generic “AI slop.”

---

0. Guiding Principles

- **Consistency over novelty.** Every generated asset must visually trace back to a locked reference — global style, then character/environment identity — rather than being reinterpreted fresh each time.
- **Isolation by default.** Every character and environment is generated in isolation (no baked-in composition), so the editor retains full control over framing, depth, and composition in post.
- **AI reasons like a director, not a transcriber.** Appearance, framing, and pacing decisions should come from contextual understanding of the story — not just literal text-matching against the script.
- **Fail cheap, not expensive.** Quality issues should be caught at the earliest, cheapest possible stage — not discovered after a full episode’s worth of assets has been generated.

---

1. Entry Point & Global Visual Style

1.1 Pipeline Entry

The pipeline supports two entry points: 1. **Title-first:** An AI model (trained specifically for this) generates a title. The title then drives AI script generation. 2. **Script-first:** If a script is already written by the user, the pipeline starts directly from that script, skipping title/script generation.

1.2 Global Visual Style — “MS Paint Style”

- A single, fixed visual style is locked for the **entire channel** — it does not vary by video or story tone.
- Style reference: a small set of reference images (not just a text description) representing the “MS Paint style”: thick black outlines, flat solid colors (no gradients/complex shading), simple/clean character proportions, and plain flat-color backgrounds.
- **Why reference images, not just text:** Text-only style descriptions drift across many separate generation calls. Reference images act as a visual anchor and must be attached to *every single generation call* in the pipeline, alongside a text description.
- **Why not fine-tune/train the style into a model:** Rejected in favor of reference-image injection because it’s more flexible (style can be adjusted anytime by swapping references), doesn’t require large training datasets, and works regardless of which underlying image model is used.

---

2. Character & Environment Extraction

2.1 Full-Script Read Requirement

AI must read and analyze the **entire script end-to-end** before doing character/environment extraction or generating any sprite sheets. This ensures assets are planned with full story context, not partial/incremental context.

2.2 Character Identification & Resolution

- AI extracts every character from the script.

- **Name/reference resolution:** AI resolves character mentions **per-instance from context** (not a static alias dictionary). Generic pronouns (“he,” “she,” “they”) are resolved situationally based on immediate context — they are never treated as fixed aliases, since the same pronoun can refer to different characters at different points in the script.
- Only **unique proper nouns/titles** are tracked in a character-name mapping (e.g., “Alexander” ↔ “the general” if that title is consistently and uniquely tied to him).

2.3 Character ID Rules

- **Named/plot-relevant characters:** Always get their own unique character ID, regardless of faction/army/side, even if visually similar to another character.
- **Unnamed background/crowd characters:** Bundled into a generic character ID — but generic IDs are scoped **per faction/side**, not shared globally. E.g., “Generic Greek Soldier” and “Generic Persian Soldier” are two distinct generic IDs, since their uniform/gear/color differs.

2.4 Character Variants (Major Visual Shifts)

A character retains **one master ID** for their entire existence in the story. Major visual discontinuities are handled as **variants** under that master ID, each with its own sprite sheet: - Age category change (child → adult → elderly) - Permanent physical state change (wounded, scarred, disguised for an extended scene, death) - Full outfit/role change that persists across a scene or act (soldier → prisoner, armor → civilian clothes)

Minor changes (helmet on/off, holding a prop, temporary pose/expression) do **not** trigger a new variant — they’re handled as different actions within the same sprite sheet.

Inference requirement: Scripts usually will not explicitly state age or outfit changes. AI must infer character appearance from **scene context** — location, time-skip signals (“years later”), and situation/activity (e.g., a character in a war scene will not be dressed as they were in a palace scene) — not just explicit descriptions.

2.5 Character Image Prompts

- Each character gets an image prompt including their **attire**, isolated on a green-screen background (see Section 6 for background-color rules).
- Prompt must specify the character is isolated with no extra elements — treated as a zoomed sprite-sheet crop.

2.6 Environment Identification & Composition

- AI extracts every distinct environment from the script.
- An environment consists of **3–4 elements** — distinct standalone objects/features within the setting (e.g., a battlefield environment’s elements: a castle, the ground/terrain, and 5–6 trees).
- **All elements of one environment are generated together in a single image** (not as separate isolated images per element). This combined image constitutes “the environment.”
- Environment images are isolated on a background per the same background-color rules as characters (Section 6).

2.7 Environment Sprite Sheets

- Each environment gets **one sprite sheet** covering the whole environment (all its elements together) — e.g., variations like season change, time-of-day change, or scene-state change (rubble after battle, etc.).
- Each environment is **enclosed in its own box** within the sprite sheet, so it’s visually clear which environment is which when multiple environments share a sheet.
- Each environment sprite sheet gets an assigned ID.

---

3. Sprite Sheets (Characters)

3.1 Action Determination

- AI determines what actions/poses a character needs by reading the **full script** (per Section 2.1) — not from a predefined base action list.
- AI has discretion to generate a fuller set of actions (up to the cap) for a character even if the script currently shows only one action for them, to provide a reasonable buffer.
- **Actions must include expression/emotion-based poses** (shocked, angry, sad, determined, neutral, etc.), not just physical-movement actions — since many beats are emotional/reaction moments, not just physical actions.

3.2 Sprite Sheet Structure

- A character can have **1 to many sprite sheets**.
- Each sprite sheet contains **2–6 actions** (hard cap: 6).
- Each sprite sheet gets an assigned unique ID.
- Grid layout (e.g., 2×3, 3×2) is left to AI discretion.

3.3 Missing-Action Fallback

If, during beat generation, a beat needs a character action/pose not covered by that character's existing sprite sheet: - AI generates a **new one-off pose** specifically for that beat, using the character's existing sprite sheet as a visual reference to maintain consistency. - This one-off pose is **not** added back into the permanent sprite sheet. - AI attaches a short note flagging that it improvised this pose, so the user can review it later if needed.

(Rejected alternatives: reusing the nearest existing action — risks visual/tonal mismatch; flagging the user manually and pausing — doesn't scale across a full episode's worth of beats.)

3.4 Quality Checkpoint (Critical Gate)

- After sprite sheets are generated (Step 3) and **before** beat generation begins (Step 4), a review checkpoint is inserted.
- Purpose: verify visual style consistency, correct background-color usage, and correct proportions/attire — because sprite sheets are the foundation every future beat-image will reference. An error here repeats across the entire episode if not caught early.
- **Beat generation must only reference sprite sheets that have passed this checkpoint.** Beats should never reference an unapproved/pending sprite sheet, to avoid downstream rework.

---

4. Beat Generation

4.1 Script → Chunks → Beats Hierarchy

- The script is first divided into **chunks**. A chunk ends at a sentence's full stop; the next chunk begins at the next sentence.
- Each chunk is then divided into **beats**.
- **A beat can never cross a chunk boundary** (chunk end = beat end).
- Chunks are a purely organizational grouping — they don't generate any separate output of their own.

4.2 Beat Length

- Beat length is roughly word-count-based and flexible depending on story/script context (example ranges discussed: ~6–14, 8–14, or 9–16 words; can be shorter).
- **When splitting a long sentence into multiple beats**, AI should prioritize natural **clause/comma/conjunction boundaries** over strict word-count cutoffs — beats must stay grammatically/semantically complete, never cutting a phrase mid-thought.

4.3 Subject/Verb/Object Breakdown (Internal Reasoning)

- For each beat, AI performs a structural breakdown (subject/main character, action, verb, generic-group representation for crowds, etc.) as part of its reasoning process.

- Example: *"He has just watched a few thousand of his countrymen shatter an army sent to enslave his city"* → Subject: he (main character) · Action: watched · Countrymen: generic group · Verb: shatter · Object: dead army (generic).
- **Default output is clean** — just the final image prompts, no breakdown shown.
- The breakdown data remains **available on demand** for review/debugging, not embedded by default in every output.

4.4 Per-Beat Prompt Generation

- For each beat, AI analyzes which characters/environment are actually needed. **Nothing is mandatory** — a beat may need only a character, only an environment, both, or neither (text-only, see 4.6).
- Each distinct element (a character, an environment/scene element) is generated as its **own isolated image prompt** — never combined into one image. Multiple elements in one beat produce multiple prompt references.
  - Example: a beat showing a character looking at a dead army produces one prompt for the character and a separate prompt for the dead-army scene.
- References combine existing sprite-sheet IDs with new prompt instances, e.g.: `S1+I1`, `S5+I2`, `E1+I3`.
- Every image prompt specifies that the character/environment is isolated with no extra elements — treated as a zoomed sprite-sheet crop.
- **Shot-type/framing variety:** AI should vary framing (close-up, wide/full-body, establishing shot) based on the beat's emotional weight and context, rather than defaulting to the same framing for every beat — for visual rhythm in the edit.
- **Multi-character image cap:** No more than 3–4 individually-detailed characters in a single generated image. Beyond that, additional figures are rendered as a generic crowd-texture/silhouette mass rather than individually detailed, since image models distort proportions/faces with too many figures in one frame.

4.5 Context Passed to AI

Each beat is passed to the AI as JSON along with long context: the full script, already-generated sprite sheets, and previous beats' prompts — so the AI always knows what has already been established. Context length is not a limiting concern given the model used.

4.6 Text-Only Beats

- Not every beat needs a visual asset. Some beats are **text-only** (just text displayed on screen, no character/environment image generated).
- These are relatively rare — roughly 4–5 occurrences in a 10-minute video.
- The output/editor handoff must clearly flag which beats are asset-based vs. text-only, so the editor knows not to expect an asset for those.
- *(Detailed sub-workflow for text-only beats to be defined separately.)*

4.7 Reference Scenes

- Certain scene types (season change, time-of-day change, character death) were initially planned to be manually authored by the user to avoid AI-driven repetition/slop.
- This was later revised: these will also be **AI-generated** during beat generation rather than manually built.
- *(Full detail of how this is handled is still to be finalized — user to define separately.)*

---

5. Reference-Chaining (Consistency Backbone)

This is a cross-cutting rule that applies throughout Steps 2–4, and is arguably the backbone of the entire pipeline's consistency:

- **Global style-kit** (Section 1.2 reference images) is attached to every generation call.

- **Additionally**, every new generation for an *existing* character or environment — a new sprite sheet, a beat-image, a variant — must also include that character/environment's own **previously-generated sprite-sheet image** as a reference input, not just a text description.
- **Why:** Text-only prompts ("character X who looks like…") produce inconsistent AI interpretations across calls, even with a style-kit present. Supplying the actual generated image as a reference gives the AI a concrete visual anchor.
- **Resulting chain:** Style-kit → Character/Environment master sprite-sheet (built from style-kit + text) → Every future generation for that asset (new sprite sheet, beat-image) references both the style-kit *and* the asset's own prior sprite-sheet image.

---

6. Background Color Rules

- **Default:** Green screen, for both characters and environments.
- **Exact green shade** should be fixed and consistently specified in every prompt (a specific standard chroma-key green) rather than left to vary per-generation.
- **Conflict fallback:** If an asset's own palette contains green (e.g., a green uniform), the background switches to **white** for that specific asset instead.
- Each asset's background color is tracked/noted in its metadata (e.g., a `background_color` field) so the editor knows which color to key for that specific asset.

---

7. Global Avoid-List

A standing "avoid list" is appended to every generation prompt across the pipeline, to protect consistency and clean keying: - No background clutter - No text/watermarks - No blurry edges - No extra unlisted characters/objects in frame - Isolated subject only

---

8. Resolution & Aspect Ratio

Videos are long-form YouTube content in **16:9**. Fixed, consistent resolutions are used across all generated assets:

Asset Type	Resolution / Ratio	Reasoning
Environment / background composites	**1920×1080 (16:9)**	Matches the final video frame directly — no resizing needed by the editor.
Character sprite sheets	**Portrait/vertical (~3:4 ratio)**	Standing figures are taller than wide; portrait orientation maximizes character detail/resolution and avoids wasted canvas space (vs. square).

---

9. Scale Consistency

Environment prompts include a **scale-reference note** (e.g., "door height ≈ human height × 1.1," "trees ≈ 3× character height") so the editor has a consistent baseline when compositing characters into environments — removing guesswork and preventing characters from looking mis-scaled relative to their surroundings (a common giveaway of "AI-generated" output).

---

10. Cross-Episode Asset Library

A shared, cross-episode asset library stores reusable generic assets — e.g., a recurring host/narrator

character, or generic crowd-types that recur across unrelated stories (a “generic modern businessman,” “generic soldier,” etc.). New videos check this library first; if a match exists, it’s reused instead of regenerated, saving cost and time as the channel scales across topics.

---

11. Scope Note: Channel Breadth

The channel is **not history-only** — it is intended to cover a broad range of topics/categories. All rules in this document (context-based appearance reasoning, environment structure, character bundling, etc.) are designed to be **genre-agnostic**, not specific to war/historical scenarios.

---

12. Explicitly Out of Scope (Handled Separately by User)

The following are acknowledged as necessary parts of the full production system but are intentionally **not** designed in this document — the user will define and manage these separately: - ID naming-scheme scoping across episodes/videos (e.g., per-episode prefixes) - Editor-handoff package format (folder structure, naming conventions, sequencing metadata) - Audio/SFX layer design (trigger timing, sourcing) - Text-only beat sub-workflow (Section 4.6) - Full reference-scene generation logic (Section 4.7)

---

13. Suggested Future Considerations (Not Yet Decided)

Raised during discussion but not yet accepted/rejected — noted here for future reference: - **Seed/generation-parameter tracking:** Saving seed, model version, and exact prompt text as metadata per asset, to improve future regeneration consistency and troubleshooting. - **Real-world trademark/logo/real-person avoidance:** A standing instruction for AI to avoid exactly reproducing real trademarked logos or real people’s likenesses when the script touches modern/business/real-world topics — to reduce copyright-strike and monetization risk. - **Draft-then-final generation strategy:** Generating a low-res/fast draft of each sprite sheet/environment first for composition/style-match review, then only generating the full-resolution final version after the draft passes — to reduce wasted cost on rejected generations. - **Cost/generation-budget tracking:** Monitoring per-video generation cost as the channel scales across many videos/topics, to keep the pipeline financially sustainable long-term.

Pipeline Summary (End-to-End Flow)

```
Title (optional, AI-generated)
   → Script (AI-generated or user-provided)
       → Global Visual Style locked (MS Paint style-kit)
           → Full script read by AI
               → Character Extraction (naming, resolution, generic bundling, variants)
               → Environment Extraction (elements combined into one image per environment)
                   → Character Sprite Sheets (actions + expressions, 2-6 per sheet, ID assigned)
                   → Environment Sprite Sheets (variations, boxed, ID assigned)
                       → QUALITY CHECKPOINT (approve before proceeding)
                           → Script divided into Chunks (sentences)
                               → Chunks divided into Beats (clause-aware, word-count guided)
                                   → Per-Beat Analysis (subject/verb/object breakdown - internal)
                                       → Per-Beat Prompt Generation (isolated multi-prompts,
                                         referencing approved sprite sheets + style-kit,
                                         shot-variety, scale-aware, text-only beats flagged)
                                           → Editor Handoff (format defined separately)
```